

Amazon ECS Cluster: Add Auto Scaling Group Later

To Begin with the Lab

Summary of the Lab

In this lab, an **Auto Scaling Group (ASG)** was created manually and then registered with an existing **Amazon ECS cluster** that had originally been created without EC2 capacity. The process started by creating the required **IAM role** for EC2, then building a **launch template** using an ECS-optimized Amazon Linux AMI, and finally creating an **Auto Scaling Group** from that launch template. After the EC2 instance launched, it registered itself with ECS and appeared in the cluster's infrastructure tab. The last step was to create a **capacity provider** for the ASG and attach it to the ECS cluster so ECS could manage scaling and lifecycle behavior automatically.

Understanding the Lab

If you create an ECS cluster without EC2 infrastructure, you can still add EC2 later. This is useful when you start with Fargate and later decide you need EC2-based workloads too. The important difference in this lab is that a simple EC2 instance is not enough for production-style ECS management. The instance must use an ECS-optimized AMI, have the correct IAM role, and be connected through an Auto Scaling Group. Registering that ASG as a capacity provider allows ECS to manage scaling, task draining, and placement more cleanly.

Lab Steps

Step 1 – Confirm the Existing ECS Cluster

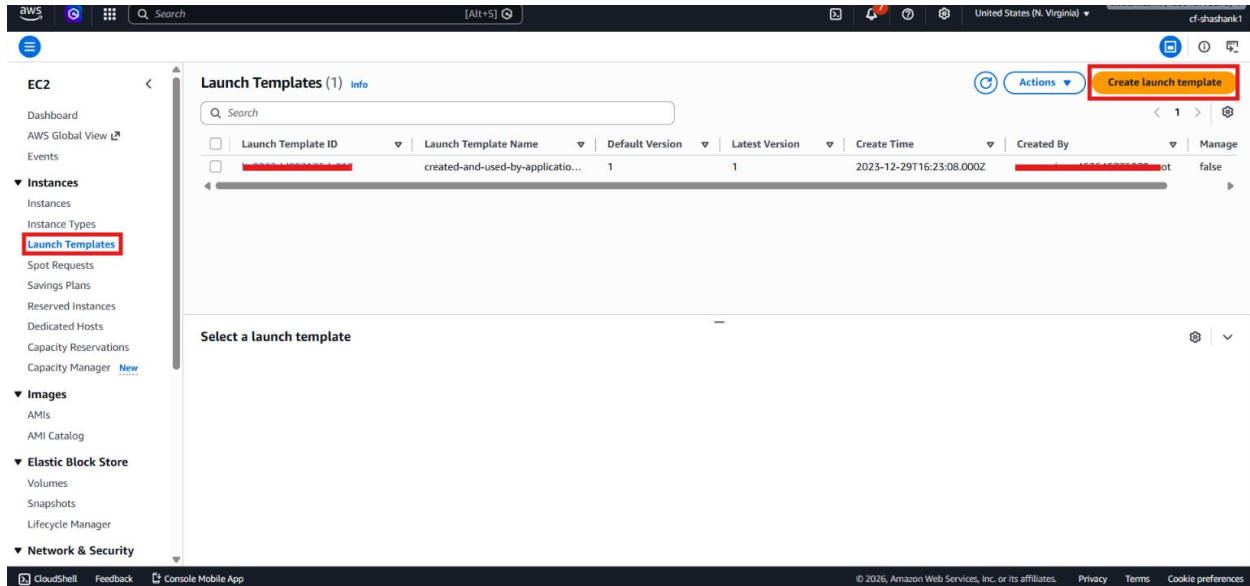
- Sign in to the AWS Management Console.
- Open **Amazon ECS**.
- Select the existing cluster I have created earlier as **MyCluster**.
- Confirm that only **Fargate** is currently available in the infrastructure tab.

Step 2 – Make Sure the IAM Role Exists

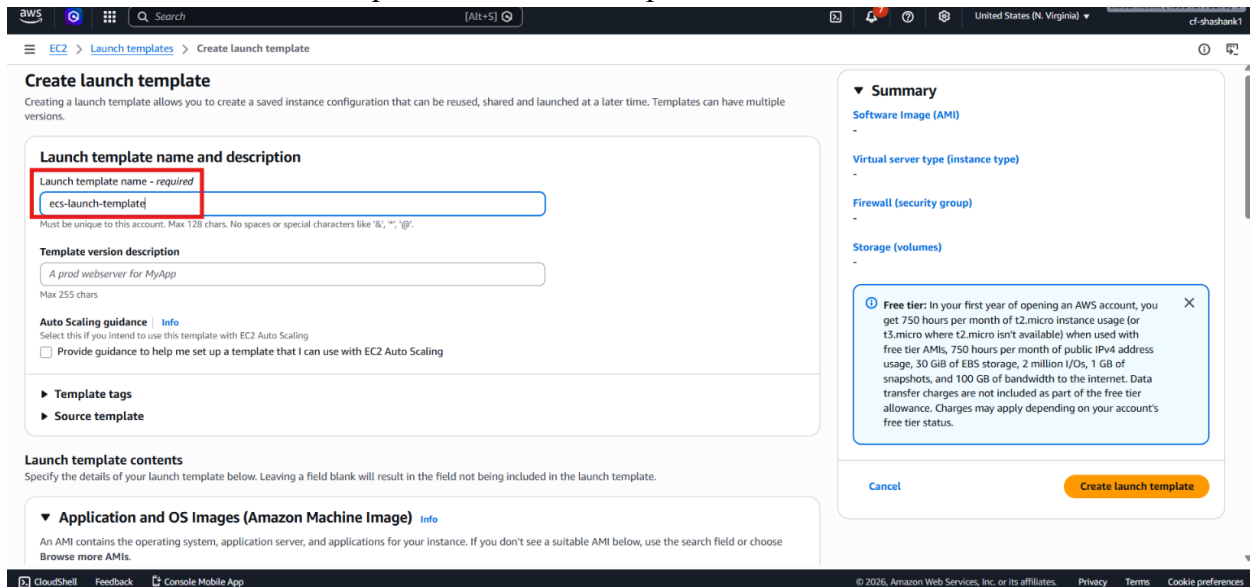
- Open **IAM**.
- Go to **Roles**.
- Confirm the role named **ecsInstanceRole** exists.
- If it does not exist, create it with:
 - Trusted entity: **EC2**
 - Policy: **AmazonEC2ContainerServiceforEC2Role**

Step 3 – Create the Launch Template

- Open Amazon EC2.
- Go to Launch Templates.
- Click Create launch template.



- Enter a name for the template: ecs-launch-template



- Select an ECS-optimized Amazon Linux AMI from the AMI Marketplace.

Amazon ECS-Optimized Amazon Linux 2 (AL2) x86_64 AMI

Sold by: [Bansir LLC](#)

Deployed on AWS | Free Trial | AWS Free Tier

This is a repackaged open source software wherein additional charges apply for extended support by Bansir. This Product contains Amazon...

4.2 ★★★★★ (46) | 2 reviews

Overview | Pricing | Legal | Usage | Resources | Support | Similar products

Overview

This is a repackaged open source software wherein additional charges apply for extended support by Bansir. This Product contains Amazon Linux that is ECS-Optimized and includes support by Bansir. Amazon Elastic Container Service is a fully managed container orchestration service. Amazon Elastic Container Service (ECS) uses Docker containers to allow you to easily run your applications on a managed cluster of ECS-

Highlights

- ECS-Optimized AMI comes pre-configured to connect to your ECS cluster
- Amazon ECS includes a scheduler for long-running applications
- Amazon ECS uses familiar AWS services such as Amazon EC2 and AWS IAM.

- Choose an instance type such as **t2.micro**.
- Select a key pair.

Instance type: **t3.micro** | Free tier eligible

The AMI vendor recommends using a t2.micro instance (or larger) for the best experience with this product.

Key pair (login)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name: **June-2026-us** | Create new key pair

Network settings

Subnet: Don't include in launch template | Create new subnet

Availability Zone: Don't include in launch template | Enable additional zones

Firewall (security groups)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Select existing security group | Create security group

Security group name - required

Summary

Software Image (AMI)
Amazon ECS-Optimized Amazon Li...read more
ami-0b52bc263a0540a40

Virtual server type (instance type)
t3.micro

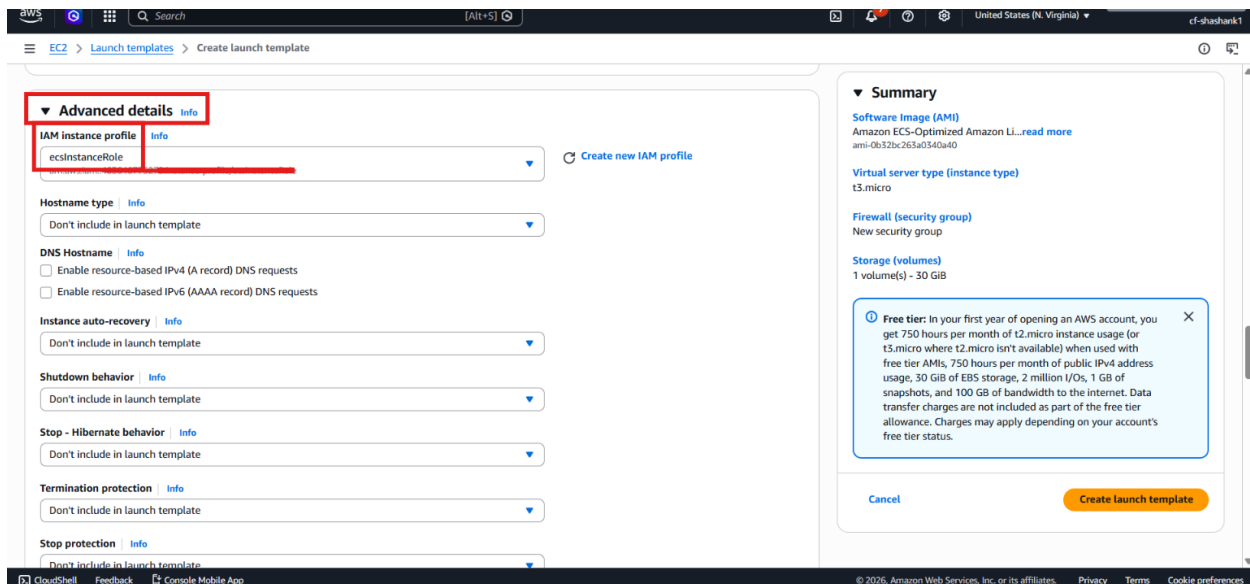
Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 30 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier allowance. Charges may apply depending on your account's free tier status.

Cancel | Create launch template

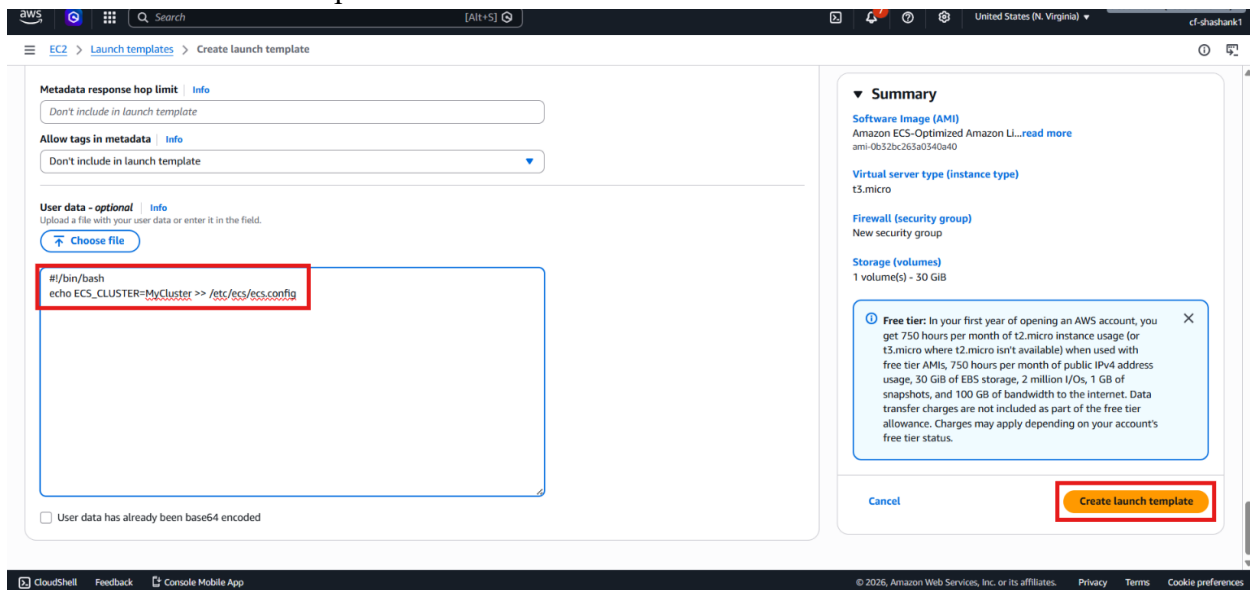
- Attach the **ecsInstanceRole** as the IAM instance profile.



- Add the cluster name in user data:

```
#!/bin/bash
echo ECS_CLUSTER=your-cluster-name >> /etc/ecs/ecs.config
```

- Create the launch template.



Step 4 – Create the Auto Scaling Group

- Open EC2 → Auto Scaling Groups.
- Click **Create Auto Scaling group**.

Amazon EC2 Auto Scaling
helps maintain the availability of your applications

Auto Scaling groups are collections of Amazon EC2 instances that enable automatic scaling and fleet management features. These features help you maintain the health and availability of your applications.

Create Auto Scaling group

Get started with EC2 Auto Scaling by [creating an Auto Scaling group](#)

Create Auto Scaling group

How it works

Auto Scaling group

Minimum size | Scale out as needed

Desired capacity

Pricing

Amazon EC2 Auto Scaling features have no additional fees beyond the service fees for Amazon EC2, CloudWatch (for scaling policies), and the other AWS resources that you use. Visit the pricing page of each service to learn more.

Getting started

[What is Amazon EC2 Auto Scaling?](#)

[Getting started with Amazon EC2 Auto Scaling](#)

- Enter the ASG name: ecs-asg
- Select the launch template created earlier.

Choose launch template

Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group.

Name

Auto Scaling group name

Enter a name to identify the group.

ecs-asg

Must be unique to this account in the current Region and no more than 255 characters.

Launch template

For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template

Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

ecs-launch-template

Create a launch template

Version

Default (1)

Create a launch template version

Description

Launch template

ecs-launch-template

Instance type

t3.micro

- Choose the default VPC.
- Select the required subnets.

Network info

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.
vpc-099792461c7d0a616
172.31.0.0/16 Default

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.
Select Availability Zones and subnets

use1-az4 (us-east-1a) subnet-0b5783ef03796c671
use1-az6 (us-east-1b) subnet-0d10d67b324b6c997

Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

☒ **Balanced best effort**
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

☐ **Balanced only**
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

☐ **Reservations Then Balanced**
Prioritizes launching into Capacity Reservations, distributing across AZs with available reservations. When reservations are fully utilized, falls back to balanced on-demand.

Additional capacity settings

- Set desired capacity to 1.
- Set maximum capacity to 2.

Group size info

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.
Units (number of instances)

Desired capacity
Specify your group size
1

Scaling info
You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity
1
Equal or less than desired capacity

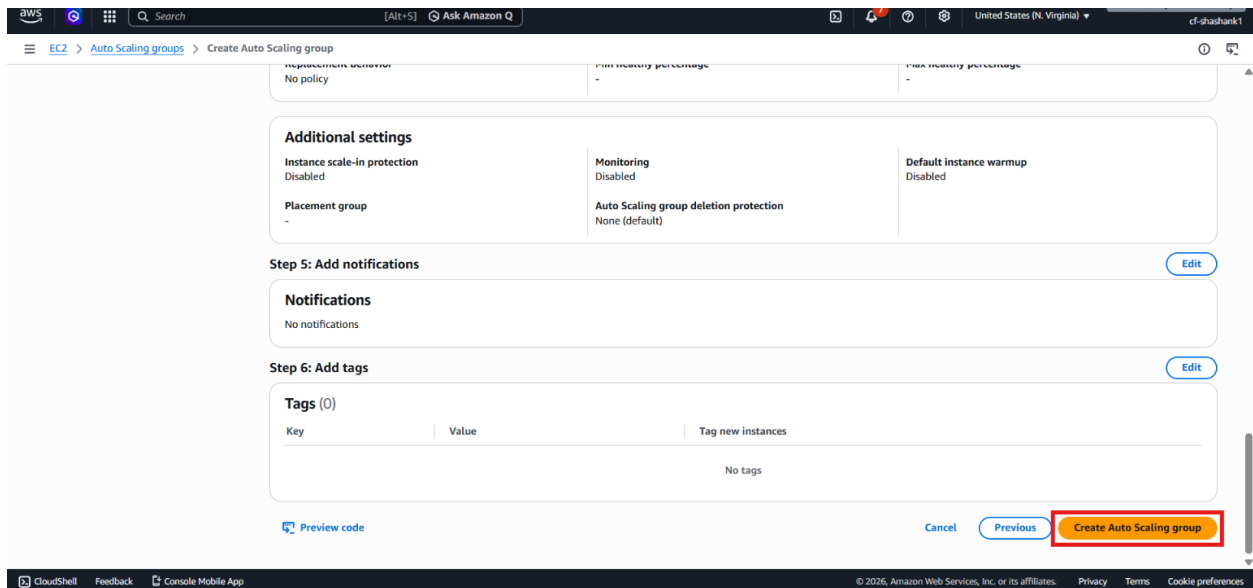
Max desired capacity
2
Equal or greater than desired capacity

Automatic scaling - optional
Choose whether to use a target tracking policy. info
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☒ **No scaling policies**
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

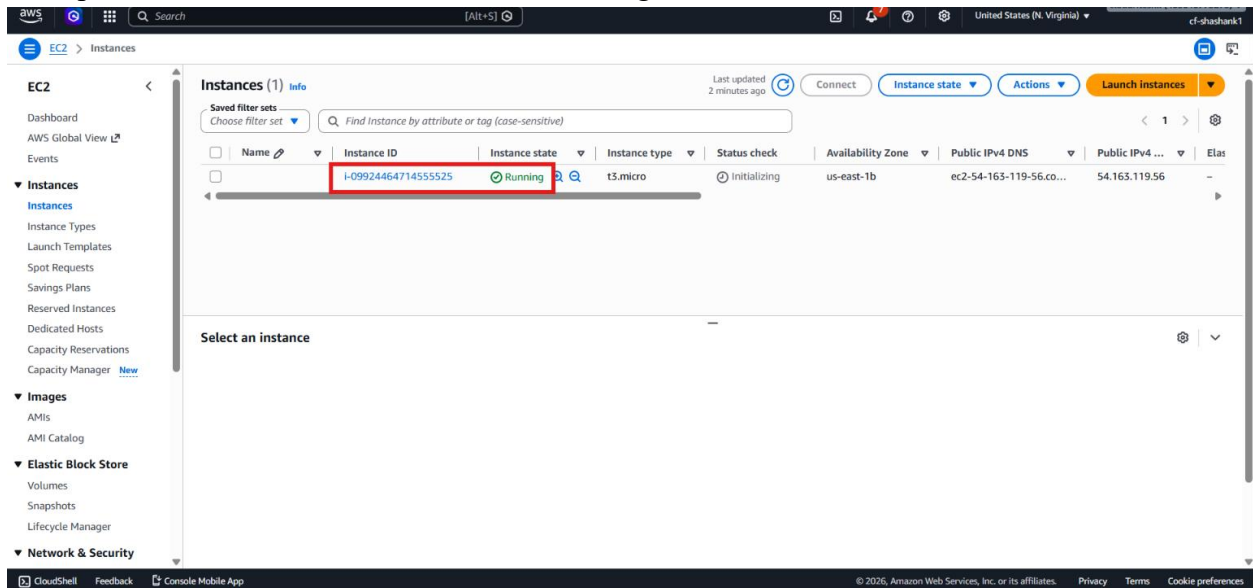
☐ **Target tracking scaling policy**
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

- Don't touch other settings, leave them to defaults and click next.
- Create the Auto Scaling Group.



Step 5 – Verify the EC2 Instance Registers with ECS

- Open EC2 and confirm the instance is running.



- Return to ECS.
- Open the cluster's **Infrastructure** tab.
- Confirm that the EC2 instance now appears as an active container instance.

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service (ECS) cluster 'MyCluster'. The 'Infrastructure' tab is selected, showing the 'Capacity providers' and 'Container instances' sections. The 'Capacity providers' section lists two providers: 'Fargate' and 'Fargate Spot'. The 'Container instances' section shows one instance with ID 'i-09924464714555525'.

Step 6 – Create a Capacity Provider

- In the ECS cluster, go to **Infrastructure**.
- Click **Create** under capacity providers.

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service (ECS) cluster 'MyCluster'. The 'Infrastructure' tab is selected, showing the 'Capacity providers' and 'Container instances' sections. The 'Capacity providers' section lists two providers: 'Fargate' and 'Fargate Spot'. The 'Container instances' section shows one instance with ID 'i-09924464714555525'.

- Select the Auto Scaling Group you created.
- Enter a capacity provider name.

Create capacity provider [Info](#)

Basic details

Cluster name
MyCluster

Capacity provider name
EC2-ASG-Provider

Scaling type
Select the capacity provider scaling type

☐ Managed Instances
Use managed instances to have Amazon ECS manage scaling and patching of instances on your behalf.

☒ EC2 Auto Scaling
Use EC2 Auto Scaling to manually configure EC2 capacity.

Scaling type cannot be updated after creation of capacity provider

Auto Scaling group [Info](#)

Use an existing Auto Scaling group
You must create an Auto Scaling group before creating a capacity provider. Use the [Amazon EC2 console](#) to create an Auto Scaling group.

ecs-asg

Scaling policies - optional [Info](#)

Manage the scaling policies for the capacity provider. Amazon ECS managed scaling uses CloudWatch metrics and a scaling policy with the Auto Scaling group to manage the scale in and scale out actions

- Create the capacity provider.

Step 7 – Attach the Capacity Provider to the Cluster

- Open the ECS cluster.
- Click **Actions** → **Update cluster**.

MyCluster

Cluster overview

Cluster ARN: arn:aws:ecs:us-east-1:123456789012:cluster/my-cluster

Status: Active

CloudWatch monitoring: Default

Services: 0 active | 0 draining

Tasks: 0 running | 0 pending

Registered container instances: 1

Services | Tasks | Daemons | **Infrastructure** | Metrics | Scheduled tasks | Configuration | Event history | Tags

Capacity providers (3) [Info](#)

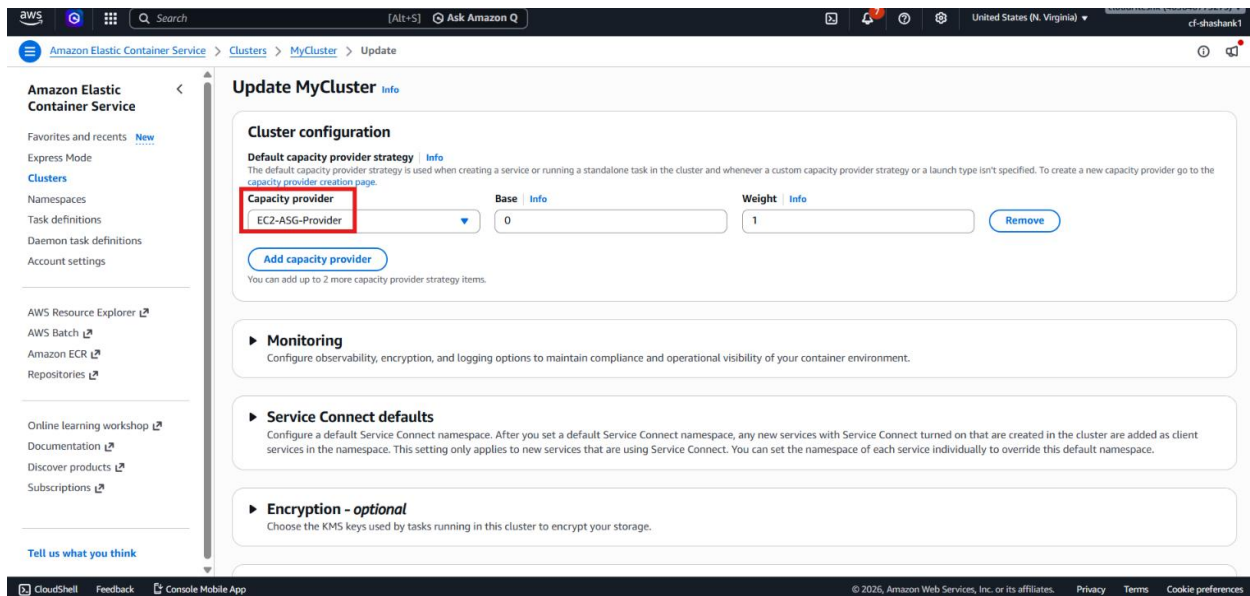
Filter capacity providers by property or value

Capacity provider	Scaling type	Provisioning model	Update status	Update status reason
☒ EC2-ASG-Provider	EC2 Auto Scaling	-	-	-
☐ Fargate	Fargate	-	-	-
☐ Fargate Spot	Fargate Spot	-	-	-

Container instances (1) [Info](#)

Filter container instances by property or value

- Add the newly created capacity provider.
- Save the update.



- Confirm that the ASG is now registered with ECS management.

Verification

- The ECS cluster was created earlier without EC2.
- The ECS instance role was created or reused.
- A launch template was created using an ECS-optimized AMI.
- An Auto Scaling Group was created successfully.
- The EC2 instance registered itself with ECS.
- A capacity provider was created for the ASG.
- The capacity provider was attached to the ECS cluster.